MODULE *Overwatch*

Section: Imports

Shared operators used by this module are imported below.

EXTENDS *Naturals*, *FiniteSets*, *Sequences*, *FailurePolicy*

Section: State and result domains

Workspace versions, completed outcomes, and message records form the result layer. Finite bounds are supplied by the *TLC* configuration, not by the runtime.

CONSTANTS *Names*, *MaxEdits*, *Timeout*, *TcrMode*

ASSUME "timeout" $\notin$ *Names* $\land$ *TcrMode* $\in$ BOOLEAN

Versions $\triangleq$ $0 \,..\, MaxEdits$

Results $\triangleq$ SUBSET *Names*

TimedOut $\triangleq$ "timeout"

Outcomes $\triangleq$ SUBSET $(Names \cup \{TimedOut\})$

NoMessage $\triangleq$ $[held \mapsto \text{FALSE},\ waking \mapsto \text{FALSE},\ failures \mapsto \{\}]$

NotTold $\triangleq$ $[kind \mapsto$ "none", $fresh \mapsto \{\}]$

Section: State shape

The checked version, unread results, inherited baseline, and delivery state are separate. *TypeOK* constrains their domains without advancing an execution.

VARIABLES
truth, *head*, *based*, *busy*, *from*, *quiet*, *installed*,
produced, *consumed*, *pending*, *report*, *baseline*, *wasFailing*,
outbox, *told*, *flush*, *turn*, *toldThisTurn*, *promptQueued*, *knows*, *elapsed*, *edited*,
tcr, *ck*, *person*, *settleDue*, *noticePending*, *noticeCause*, *noticeTold*

vars $\triangleq$ $\langle truth, head, based, busy, from, quiet, installed,$
$produced, consumed, pending, report, baseline, wasFailing,$
$outbox, told, flush, turn, toldThisTurn, promptQueued, knows, elapsed, edited,$
$tcr, ck, person, settleDue, noticePending, noticeCause, noticeTold\rangle$

tcrVars $\triangleq$ $\langle tcr, ck, person, settleDue, noticePending, noticeCause, noticeTold\rangle$

TypeOK $\triangleq$
$\land$ $truth \in [Versions \to Results]$
$\land$ $head \in Versions \land based \in 0 \,..\, head$
$\land$ $busy \in$ BOOLEAN $\land from \in Versions \land quiet \in$ BOOLEAN $\land installed \in$ BOOLEAN
$\land$ $produced \in 0 \,..\, (MaxEdits + 1) \land consumed \in 0 \,..\, produced$
$\land$ $pending \in Seq([id : 1 \,..\, (MaxEdits + 1),\ at : Versions,\ failures : Outcomes])$
$\land$ $Len(pending) \leq MaxEdits + 1$
$\land$ $report \in [known :$ BOOLEAN $,\ id : 0 \,..\, produced,\ at : Versions,\ failures : Outcomes]$
$\land$ $baseline \in [known :$ BOOLEAN $,\ failures : Outcomes]$
$\land$ $wasFailing \in$ BOOLEAN
$\land$ $outbox \in [held :$ BOOLEAN $,\ waking :$ BOOLEAN $,\ failures : Outcomes]$
$\land$ $told \in [kind : \{$"none", "failed", "recovered"$\},\ fresh : Outcomes]$
$\land$ $flush \in$ BOOLEAN
$\land$ $turn \in \{$"idle", "running"$\}$
$\land$ $toldThisTurn \in$ BOOLEAN $\land promptQueued \in$ BOOLEAN
$\land$ $knows \in Outcomes \land elapsed \in 0 \,..\, Timeout$
$\land$ $edited \in$ BOOLEAN
$\land$ $tcr \in \{$"off", "active", "unavailable"$\} \land ck \in Versions \land ck \leq head$
$\land$ $person \in \{$"none", "unseen", "seen"$\} \land settleDue \in$ BOOLEAN
$\land$ $noticePending \in$ BOOLEAN $\land noticeCause \in Outcomes \land noticeTold \in Outcomes$

Section: Failure gating

A failure report becomes a gate only when it is a regression against the session baseline, or when the check times out.

$Failing \triangleq report.known \land report.failures \neq \{\}$
$Regressed(f) \triangleq TimedOut \in f \lor (baseline.known \land NewFailure(f,\ baseline.failures))$
$Guard \triangleq Failing \land Regressed(report.failures)$
$Clean \triangleq person = \text{“none”} \land ck = head$

Section: Initial state

The first check starts at the inherited workspace version. No result or baseline is known and no notification has been sent.

$Init \triangleq$
$\land truth \in [Versions \to Results]$
$\land head = 0 \land based = 0$
$\land busy = \text{TRUE} \land from = 0 \land quiet = \text{FALSE} \land installed = \text{FALSE}$
$\land produced = 0 \land consumed = 0 \land pending = \langle\rangle$
$\land report = [known \mapsto \text{FALSE},\ id \mapsto 0,\ at \mapsto 0,\ failures \mapsto \{\}]$
$\land baseline = [known \mapsto \text{FALSE},\ failures \mapsto \{\}]$
$\land wasFailing = \text{FALSE}$
$\land outbox = NoMessage \land told = NotTold \land flush = \text{FALSE}$
$\land turn = \text{“idle”} \land toldThisTurn = \text{FALSE} \land promptQueued = \text{FALSE}$
$\land knows = \{\} \land elapsed = 0 \land edited = \text{FALSE}$
$\land person \in \text{IF } TcrMode \text{ THEN } \{\text{“none”}, \text{“seen”}\} \text{ ELSE } \{\text{“none”}\}$
$\land tcr = \text{IF } \neg TcrMode \text{ THEN “off” ELSE IF } person = \text{“none” THEN “active” ELSE “unavailable”}$
$\land ck = 0 \land settleDue = \text{FALSE}$
$\land noticePending = \text{FALSE} \land noticeCause = \{\} \land noticeTold = \{\}$

Section: Result delivery and baseline isolation

Queue deduplicates notification signatures. Deliver consumes a completed snapshot, establishes the inherited baseline, and queues a regression or recovery. A recovery can cancel a failure message that has not yet been delivered.

$Queue(signature,\ message) \triangleq$
IF $told = signature \land \neg outbox.held$
THEN UNCHANGED $\langle outbox,\ told,\ flush\rangle$
ELSE $\land told' = signature \land outbox' = message \land flush' = \text{TRUE}$

$Deliver(v,\ id,\ f) \triangleq$
LET $b \triangleq \text{IF } baseline.known \text{ THEN } baseline.failures \text{ ELSE IF } head = based \text{ THEN } f \text{ ELSE } \{\}$
$isRegression \triangleq TimedOut \in f \lor \neg(f \subseteq b)$
IN $\land report' = [known \mapsto \text{TRUE},\ id \mapsto id,\ at \mapsto v,\ failures \mapsto f]$
$\land baseline' = \text{IF } TimedOut \in f \text{ THEN } baseline \text{ ELSE } [known \mapsto \text{TRUE},\ failures \mapsto b]$
$\land wasFailing' = isRegression$
$\land$ IF $isRegression$
THEN $Queue([kind \mapsto \text{“failed”},\ fresh \mapsto f \setminus b],$
$[held \mapsto \text{TRUE},\ waking \mapsto edited \lor TimedOut \in f,\ failures \mapsto f])$
ELSE IF $\neg wasFailing$
THEN UNCHANGED $\langle outbox,\ told,\ flush\rangle$
ELSE IF $outbox.held$
THEN $\land outbox' = NoMessage \land told' = NotTold \land$ UNCHANGED $flush$
ELSE $Queue([kind \mapsto \text{“recovered”},\ fresh \mapsto \{\}],\ [held \mapsto \text{TRUE},\ waking \mapsto \text{FALSE},\ failures \mapsto f])$

Section: Workspace changes and scheduling

An agent edit advances head without moving based, and marks the next read as following the agent's own change. Another process can change the workspace at any time without committing; *OutsideEdit* advances head the same way but marks nothing. An external head change moves both and invalidates the baseline and unread results. *QuietElapses* starts the next check; *Tick* advances its timeout clock.

$Edit \triangleq$
$\land turn = \text{“running”} \land head < MaxEdits$
$\land head' = head + 1 \land quiet' = \text{TRUE} \land edited' = \text{TRUE}$
$\land settleDue' = (settleDue \lor tcr = \text{“active”})$
$\land$ UNCHANGED $\langle truth,\ based,\ busy,\ from,\ installed,\ produced,\ consumed,\ pending,$
$report,\ baseline,\ wasFailing,\ outbox,\ told,\ flush,\ turn,$

```
                    toldThisTurn, promptQueued, knows, elapsed, tcr, ck, person,
                    noticePending, noticeCause, noticeTold⟩

OutsideEdit ≜
    ∧ head < MaxEdits
    ∧ head′ = head + 1 ∧ quiet′ = TRUE
    ∧ person′ = IF TcrMode ∧ person = "none" THEN "unseen" ELSE person
    ∧ UNCHANGED ⟨truth, based, busy, from, installed, produced, consumed, pending,
                 report, baseline, wasFailing, outbox, told, flush, turn,
                 toldThisTurn, promptQueued, knows, elapsed, edited, tcr, ck, settleDue,
                 noticePending, noticeCause, noticeTold⟩

HeadMoves ≜
    ∧ head < MaxEdits
    ∧ TcrMode ⇒ (person = "none" ∧ ck = head)
    ∧ head′ = head + 1 ∧ based′ = head + 1
    ∧ ck′ = IF TcrMode THEN head + 1 ELSE ck
    ∧ baseline′ = [known ↦ FALSE, failures ↦ {}]
    ∧ quiet′ = TRUE ∧ pending′ = ⟨⟩
    ∧ from′ = IF busy THEN head + 1 ELSE from
    ∧ elapsed′ = IF busy THEN 0 ELSE elapsed
    ∧ UNCHANGED ⟨truth, busy, installed, produced, consumed, report, wasFailing, outbox,
                 told, flush, turn, toldThisTurn, promptQueued, knows, edited, tcr, person,
                 settleDue, noticePending, noticeCause, noticeTold⟩

QuietElapses ≜
    ∧ quiet ∧ installed
    ∧ quiet′ = FALSE ∧ busy′ = TRUE ∧ from′ = head ∧ elapsed′ = 0
    ∧ UNCHANGED ⟨truth, head, based, installed, produced, consumed, pending,
                 report, baseline, wasFailing, outbox, told, flush, turn,
                 toldThisTurn, promptQueued, knows, edited, tcrVars⟩

Tick ≜
    ∧ busy ∧ elapsed < Timeout ∧ elapsed′ = elapsed + 1
    ∧ UNCHANGED ⟨truth, head, based, busy, from, quiet, installed, produced,
                 consumed, pending, report, baseline, wasFailing, outbox,
                 told, flush, turn, toldThisTurn, promptQueued, knows, edited, tcrVars⟩
```

Completion appends a result; it does not deliver it. Partial output is a stuttering step at this abstraction: only a complete build enters pending.

Section: Buffered completion and consumption

Run completion appends a snapshot to pending without publishing it. *ReadResults* consumes the newest completed snapshot and clears the buffer. A later incomplete build does not replace an earlier complete result.

```
Buffer(v, f) ≜
    ∧ produced′ = produced + 1
    ∧ pending′ = Append(pending, [id ↦ produced + 1, at ↦ v, failures ↦ f])
    ∧ busy′ = FALSE ∧ installed′ = TRUE
    ∧ UNCHANGED ⟨truth, head, based, from, quiet, consumed, report, baseline,
                 wasFailing, outbox, told, flush, turn, toldThisTurn, promptQueued, knows, elapsed, edited,
                 tcrVars⟩

RunEnds ≜ busy ∧ ∃ v ∈ from .. head : Buffer(v, truth[v])
TimesOut ≜ busy ∧ elapsed = Timeout ∧ Buffer(from, {TimedOut})

ReadResults ≜
    ∧ pending ≠ ⟨⟩
    ∧ LET latest ≜ pending[Len(pending)]
      IN  Deliver(latest.at, latest.id, latest.failures)
```

```
193     ∧ consumed′ = produced ∧ pending′ = ⟨⟩ ∧ edited′ = FALSE
194     ∧ UNCHANGED ⟨truth, head, based, busy, from, quiet, installed, produced,
195                  turn, toldThisTurn, promptQueued, knows, elapsed, tcrVars⟩
```

Section: Notification delivery and turn completion

A running turn receives at most one notification before another turn starts. An idle agent can receive one queued wakeup. Flush publishes eligible messages; *TurnCompletes* keeps a guarded turn running and permits a clean turn to settle.

```
203 Held ≜ IF turn = "running" THEN toldThisTurn ELSE promptQueued ∨ ¬outbox.waking

205 Flush ≜
206     ∧ flush ∧ flush′ = FALSE
207     ∧ IF outbox.held ∧ ¬Held
208         THEN ∧ outbox′ = NoMessage ∧ knows′ = outbox.failures
209              ∧ IF turn = "running"
210                   THEN toldThisTurn′ = TRUE ∧ UNCHANGED promptQueued
211                   ELSE promptQueued′ = TRUE ∧ UNCHANGED toldThisTurn
212         ELSE UNCHANGED ⟨outbox, knows, toldThisTurn, promptQueued⟩
213     ∧ UNCHANGED ⟨truth, head, based, busy, from, quiet, installed, produced, consumed,
214                  pending, report, baseline, wasFailing, told, turn, elapsed, edited, tcrVars⟩

216 TurnStarts ≜
217     ∧ turn = "idle" ∧ turn′ = "running"
218     ∧ toldThisTurn′ = promptQueued ∧ promptQueued′ = FALSE ∧ flush′ = TRUE
219     ∧ tcr′ = IF tcr = "off" THEN "off" ELSE IF Clean THEN "active" ELSE "unavailable"
220     ∧ UNCHANGED ⟨truth, head, based, busy, from, quiet, installed, produced, consumed,
221                  pending, report, baseline, wasFailing, outbox, told, knows, elapsed, edited,
222                  ck, person, settleDue, noticePending, noticeCause, noticeTold⟩

224 WokenTurnStarts ≜ promptQueued ∧ TurnStarts

226 TurnCompletes ≜
227     ∧ turn = "running"
228     ∧ IF Guard
229         THEN ∧ toldThisTurn′ = TRUE ∧ knows′ = report.failures ∧ UNCHANGED ⟨turn, flush⟩
230         ELSE ∧ turn′ = "idle" ∧ flush′ = TRUE ∧ UNCHANGED ⟨toldThisTurn, knows⟩
231     ∧ UNCHANGED ⟨truth, head, based, busy, from, quiet, installed, produced, consumed,
232                  pending, report, baseline, wasFailing, outbox, told, promptQueued, elapsed, edited,
233                  tcrVars⟩
```

Section: *TCR* mode

With *TcrMode*, an edit of the agent is settled once a result for the current workspace has been read: a green result commits the workspace as the new restore point *ck*, and a result with a new failure restores the workspace to the restore point. A revert is a new workspace version, so head only grows, and its content is that of the restore point. A pull replaces the restore point. A person's uncommitted work is unseen after an outside edit and seen once a tool that does not edit follows it, or when it was there at the start; *TCR* then waits until the person commits it. A check that timed out cannot judge an edit. The notice of a revert is kept apart from the failure message, so the recovery that a revert causes cannot cancel it. A pull with uncommitted work and the instant in which a hook edits and settles are outside this abstraction: a turn may start before the result of an edit is read, and *TCR* then waits.

```
250 Judged ≜ tcr = "active" ∧ settleDue ∧ report.known ∧ report.at = head
251            ∧ TimedOut ∉ report.failures
252 FreshFailures ≜ report.failures \ baseline.failures

254 Observe ≜
255     ∧ tcr = "active" ∧ person = "unseen" ∧ ¬settleDue
256     ∧ person′ = "seen" ∧ tcr′ = "unavailable"
257     ∧ UNCHANGED ⟨truth, head, based, busy, from, quiet, installed, produced, consumed,
258                  pending, report, baseline, wasFailing, outbox, told, flush, turn,
259                  toldThisTurn, promptQueued, knows, elapsed, edited, ck, settleDue,
260                  noticePending, noticeCause, noticeTold⟩
```

$PersonCommits \triangleq$
  $\land\ tcr = \text{“unavailable”}$
  $\land\ ck' = head \land person' = \text{“none”} \land settleDue' = \text{FALSE}$
  $\land$ UNCHANGED $\langle truth, head, based, busy, from, quiet, installed, produced, consumed,$
    $pending, report, baseline, wasFailing, outbox, told, flush, turn,$
    $toldThisTurn, promptQueued, knows, elapsed, edited, tcr,$
    $noticePending, noticeCause, noticeTold\rangle$

$Checkpoint \triangleq$
  $\land\ Judged \land \neg Guard$
  $\land\ ck' = head \land settleDue' = \text{FALSE} \land person' = \text{“none”}$
  $\land$ UNCHANGED $\langle truth, head, based, busy, from, quiet, installed, produced, consumed,$
    $pending, report, baseline, wasFailing, outbox, told, flush, turn,$
    $toldThisTurn, promptQueued, knows, elapsed, edited, tcr,$
    $noticePending, noticeCause, noticeTold\rangle$

$CheckpointFails \triangleq$
  $\land\ Judged \land \neg Guard$
  $\land\ tcr' = \text{“unavailable”}$
  $\land$ UNCHANGED $\langle truth, head, based, busy, from, quiet, installed, produced, consumed,$
    $pending, report, baseline, wasFailing, outbox, told, flush, turn,$
    $toldThisTurn, promptQueued, knows, elapsed, edited, ck, person,$
    $settleDue, noticePending, noticeCause, noticeTold\rangle$

$Revert \triangleq$
  $\land\ Judged \land Guard \quad \land person \neq \text{“seen”}$
  $\land\ head < MaxEdits \land truth[head + 1] = truth[ck]$
  $\land\ head' = head + 1 \land ck' = head + 1 \land quiet' = \text{TRUE}$
  $\land\ settleDue' = \text{FALSE} \land person' = \text{“none”}$
  $\land\ noticePending' = \text{TRUE}$
  $\land\ noticeCause' = \text{IF } noticePending \text{ THEN } noticeCause \cup FreshFailures \text{ ELSE } FreshFailures$
  $\land$ UNCHANGED $\langle truth, based, busy, from, installed, produced, consumed, pending,$
    $report, baseline, wasFailing, outbox, told, flush, turn, toldThisTurn,$
    $promptQueued, knows, elapsed, edited, tcr, noticeTold\rangle$

$TellRevert \triangleq$
  $\land\ noticePending$
  $\land\ noticePending' = \text{FALSE} \land noticeTold' = noticeCause$
  $\land$ UNCHANGED $\langle truth, head, based, busy, from, quiet, installed, produced, consumed,$
    $pending, report, baseline, wasFailing, outbox, told, flush, turn,$
    $toldThisTurn, promptQueued, knows, elapsed, edited, tcr, ck, person,$
    $settleDue, noticeCause\rangle$

Section: Allowed executions and fairness

Next chooses an enabled transition. Spec permits stuttering and requires weak fairness for the listed progress actions; this does not impose deadlines.

$Next \triangleq\ Edit \lor OutsideEdit \lor HeadMoves \lor QuietElapses \lor Tick \lor RunEnds \lor TimesOut$
  $\lor\ ReadResults \lor Flush \lor TurnStarts \lor TurnCompletes$
  $\lor\ Observe \lor PersonCommits \lor Checkpoint \lor CheckpointFails \lor Revert \lor TellRevert$

$Spec \triangleq$
  $\land\ Init \land \Box[Next]_{vars}$
  $\land\ \text{WF}_{vars}(QuietElapses) \land \text{WF}_{vars}(Tick) \land \text{WF}_{vars}(TimesOut)$
  $\land\ \text{WF}_{vars}(ReadResults) \land \text{WF}_{vars}(Flush)$
  $\land\ \text{WF}_{vars}(WokenTurnStarts) \land \text{WF}_{vars}(TurnCompletes) \land \text{WF}_{vars}(TellRevert)$

Section: Safety and progress properties

Results retain their ordering and checked version. The baseline cannot absorb later edits. A failure wakes the agent only when its read follows the agent's own edit, or when the check timed out, so another process's changes never wake it. Under *Spec* fairness, checks end, and each regression reaches the agent or waits, without waking it, for its next turn. These properties are model-checked within the committed configuration bounds.

$$BufferedResultsAreOrdered \triangleq$$
$$\quad \land \forall\, i \in 1 \,..\, Len(pending) : pending[i].id > consumed \land pending[i].id \leq produced$$
$$\quad \land \forall\, i \in 1 \,..\, (Len(pending) - 1) : pending[i].id < pending[i+1].id$$

$$ReadsSelectLatestComplete \triangleq report.known \Rightarrow report.id = consumed$$
$$ResultsMatchBuilds \triangleq report.known \Rightarrow report.failures = truth[report.at] \lor report.failures = \{TimedOut\}$$
$$BaselineNeverAbsorbsEdits \triangleq baseline.known \Rightarrow baseline.failures \subseteq truth[based]$$
$$RunsEnd \triangleq busy \leadsto \neg busy$$
$$EveryEditIsChecked \triangleq \Diamond\Box(report.known \land report.at = head)$$
$$Regressions \triangleq truth[head] \setminus truth[based]$$
$$WaitsForTurn \triangleq turn = \text{“idle”} \land outbox.held \land \neg outbox.waking \land Regressions \subseteq outbox.failures$$
$$RegressionsReachAgent \triangleq \Diamond\Box(Regressions \subseteq knows \lor TimedOut \in knows \lor WaitsForTurn)$$
$$OutsideChangesNeverWake \triangleq$$
$$\quad \Box[told' \neq told \land told'.kind = \text{“failed”} \land outbox'.waking$$
$$\qquad \Rightarrow edited \lor TimedOut \in outbox'.failures]_{vars}$$

Section: *TCR* properties

A revert never discards work of a person that was there at the start or that a tool that does not edit has seen. Work of a person that no such tool has seen, because it came just before the agent's edit, cannot be told from the agent's and a revert discards it: this is a limit, not a property. A commit settles only a green result of the latest edit, a revert only a result with a new failure of the latest edit, and neither acts on an overtaken result, a check that timed out, or while *TCR* is unavailable. A failed commit changes nothing, so no work is lost. The restore point is never older than the baseline, and the notice of every revert reaches the agent.

$$SeenWorkPausesTcr \triangleq person = \text{“seen”} \Rightarrow tcr \neq \text{“active”}$$
$$RestorePointIsNotOlderThanBaseline \triangleq ck \geq based$$
$$NoticeNamesNewFailures \triangleq noticePending \Rightarrow noticeCause \neq \{\} \land TimedOut \notin noticeCause$$
$$HeadOnlyGrows \triangleq \Box[head' \geq head]_{vars}$$
$$RevertNeverDiscardsSeenWork \triangleq \Box[Revert \Rightarrow person \neq \text{“seen”}]_{vars}$$
$$RevertRestoresCheckpointContent \triangleq \Box[Revert \Rightarrow truth[head'] = truth[ck]]_{vars}$$
$$RevertFollowsNewFailure \triangleq \Box[Revert \Rightarrow NewFailure(report.failures,\ baseline.failures)]_{vars}$$
$$CheckpointFollowsGreenResult \triangleq$$
$$\quad \Box[Checkpoint \Rightarrow report.known \land \neg Guard \land TimedOut \notin report.failures]_{vars}$$
$$OvertakenResultsNeverSettle \triangleq$$
$$\quad \Box[(Checkpoint \lor CheckpointFails \lor Revert) \Rightarrow report.at = head]_{vars}$$
$$UnavailableTcrChangesNothing \triangleq$$
$$\quad \Box[tcr \neq \text{“active”} \Rightarrow \neg(Checkpoint \lor CheckpointFails \lor Revert)]_{vars}$$
$$FailedCommitLosesNothing \triangleq$$
$$\quad \Box[CheckpointFails \Rightarrow head' = head \land ck' = ck \land person' = person \land settleDue' = settleDue]_{vars}$$
$$RestorePointMovesWithCommits \triangleq$$
$$\quad \Box[ck' \neq ck \Rightarrow (Checkpoint \lor Revert \lor HeadMoves \lor PersonCommits)]_{vars}$$
$$RevertRaisesNotice \triangleq \Box[Revert \Rightarrow noticePending']_{vars}$$
$$RevertNoticeTellsTheCause \triangleq$$
$$\quad \Box[TellRevert \Rightarrow noticeTold' = noticeCause \land noticeTold' \neq \{\}]_{vars}$$
$$RevertNoticeIsNeverDropped \triangleq \Box[(noticePending \land \neg noticePending') \Rightarrow TellRevert]_{vars}$$
$$RevertNoticeReachesAgent \triangleq noticePending \leadsto \neg noticePending$$